

Assignment 2

Part A: 1NF, 2NF, 3NF

Task 1 - First Normal Form (1NF)

Original Student Table:

StudentID	StudentName	CoursesEnrolled	PhoneNumbers
101	Alice Smith	DBMS, OOP, Networks	123-4567, 987-6543
102	Bob Brown	DBMS, AI	555-2222
103	Carol White	OOP, Networks, Security	888-1111, 444-3333

1NF Violations

1NF requires: (1) each column holds only atomic values, (2) no repeating groups, (3) a primary key to uniquely identify each row.

Two columns violate 1NF:

- CoursesEnrolled stores multiple course names in a single cell (e.g., "DBMS, OOP, Networks"). 1NF requires each cell to hold exactly one atomic value.
- PhoneNumbers also stores multiple phone numbers in a single cell (e.g., "123-4567, 987-6543"). This is another multi-valued, non-atomic attribute.

Normalized to 1NF

To satisfy 1NF we expand the multi-valued attributes into separate rows. The composite primary key becomes (StudentID, CourseEnrolled, PhoneNumber).

StudentID	StudentName	CourseEnrolled	PhoneNumber
101	Alice Smith	DBMS	123-4567
101	Alice Smith	DBMS	987-6543
101	Alice Smith	OOP	123-4567
101	Alice Smith	OOP	987-6543
101	Alice Smith	Networks	123-4567
101	Alice Smith	Networks	987-6543
102	Bob Brown	DBMS	555-2222
102	Bob Brown	AI	555-2222
103	Carol White	OOP	888-1111

StudentID	StudentName	CourseEnrolled	PhoneNumber
103	Carol White	OOP	444-3333
103	Carol White	Networks	888-1111
103	Carol White	Networks	444-3333
103	Carol White	Security	888-1111
103	Carol White	Security	444-3333

Each cell now holds a single atomic value. Every row is uniquely identified by the composite key (StudentID, CourseEnrolled, PhoneNumber). This eliminates all repeating groups and multi-valued fields.

Task 2 - Second Normal Form (2NF)

Original Enrollment Table:

StudentID	Major	CourseID	CourseTitle	InstructorID	InstructorName	InstructorOffice	Grade
101	CS	DBMS101	Database Systems	I11	Dr. Khan	Room 304	A
101	CS	OOP201	OOP in Java	I12	Prof. Lee	Room 210	B+
102	IT	DBMS101	Database Systems	I11	Dr. Khan	Room 304	A-
103	CS	NET150	Computer Networks	I13	Dr. Silva	Room 118	B
102	IT	NET150	Computer Networks	I13	Dr. Silva	Room 118	B+

Primary Key Identification

Primary Key: (StudentID, CourseID)

A student can enroll in many courses, and a course can have many students (many-to-many relationship). Neither StudentID alone nor CourseID alone can uniquely identify a row. The combination (StudentID, CourseID) is the minimal set that uniquely identifies each enrollment record.

Functional Dependencies

FD1: (StudentID, CourseID) → Grade

FD2: StudentID → Major

FD3: CourseID → CourseTitle

FD4: CourseID → InstructorID

FD5: InstructorID → InstructorName

FD6: InstructorID → InstructorOffice

FD1 is a full functional dependency on the composite key. FD2 to FD6 are partial dependencies. They depend on only part of the composite primary key which violates 2NF.

Decomposition into 2NF

We remove all partial dependencies by splitting the table into three relations:

Relation 1 - Enrollment: (StudentID, CourseID, Grade)

Primary Key: (StudentID, CourseID)

StudentID	CourseID	Grade
101	DBMS101	A
101	OOP201	B+
102	DBMS101	A-
103	NET150	B
102	NET150	B+

Relation 2 - Student: (StudentID, Major)

Primary Key: StudentID

StudentID	Major
101	CS
102	IT
103	CS

Relation 3 - Course: (CourseID, CourseTitle, InstructorID, InstructorName, InstructorOffice)

Primary Key: CourseID

CourseID	CourseTitle	InstructorID	InstructorName	InstructorOffice
DBMS101	Database Systems	I11	Dr. Khan	Room 304
OOP201	OOP in Java	I12	Prof. Lee	Room 210
NET150	Computer Networks	I13	Dr. Silva	Room 118

The partial dependencies ($\text{StudentID} \rightarrow \text{Major}$) and ($\text{CourseID} \rightarrow \text{CourseTitle}, \text{InstructorID}, \text{InstructorName}, \text{InstructorOffice}$) have been moved to their own relations, each with the appropriate single-column primary key. The Enrollment relation now contains only the full functional dependency on the composite key.

Task 3 - Third Normal Form (3NF)

Original Employee Table:

EmpID	EmpName	DeptID	DeptName	DeptPhone
201	Alice Green	D01	Computer Science	555-1111
202	Bob Brown	D02	Mathematics	555-2222
203	Carol White	D01	Computer Science	555-1111
204	David Khan	D03	Physics	555-3333
205	Eva Silva	D02	Mathematics	555-2222

Non-Trivial Functional Dependencies

FD1: EmpID → EmpName, DeptID, DeptName, DeptPhone

FD2: EmpID → EmpName

FD3: EmpID → DeptID

FD4: DeptID → DeptName

FD5: DeptID → DeptPhone

The primary key is EmpID. FD1 shows that EmpID determines all other attributes. However, FD4 and FD5 reveal that DeptName and DeptPhone are determined by DeptID which is a non-key attribute. This creates a transitive dependency:

EmpID → DeptID → DeptName

EmpID → DeptID → DeptPhone

These transitive dependencies violate 3NF (which requires that no non-key attribute be transitively dependent on the primary key).

Decomposition into 3NF

We split the table into two relations, eliminating the transitive dependency:

Relation 1 — Employee: (EmpID, EmpName, DeptID)

Primary Key: EmpID

Foreign Key: DeptID → Department

EmpID	EmpName	DeptID
201	Alice Green	D01
202	Bob Brown	D02
203	Carol White	D01
204	David Khan	D03
205	Eva Silva	D02

Relation 2 — Department: (DeptID, DeptName, DeptPhone)

Primary Key: DeptID

DeptID	DeptName	DeptPhone
D01	Computer Science	555-1111
D02	Mathematics	555-2222
D03	Physics	555-3333

Both relations are in 3NF because: (1) they are already in 2NF (each has a single-column key, so no partial dependencies exist), and (2) every non-key attribute depends directly on the primary key only; there are no transitive dependencies remaining.

Part B: BCNF, 4NF, 5NF

Task 1 - Boyce-Codd Normal Form (BCNF)

Original Table - Student-Course-Instructor:

StudentID	CourseID	InstructorID
101	DBMS101	I11
101	OOP201	I12
102	DBMS101	I11
103	NET150	I13
104	OOP201	I12

Functional Dependencies

Assumptions derived from the data:

- Each course section is taught by exactly one instructor: CourseID $\rightarrow$ InstructorID
- A student enrolled in a course is taught by that course's instructor: (StudentID, CourseID) $\rightarrow$ InstructorID
- An instructor teaches only one course: InstructorID $\rightarrow$ CourseID

FD1: CourseID $\rightarrow$ InstructorID**FD2:** (StudentID, CourseID) $\rightarrow$ InstructorID (follows from FD1)**FD3:** InstructorID $\rightarrow$ CourseID

BCNF Analysis

Candidate keys: Both (StudentID, CourseID) and (StudentID, InstructorID) are candidate keys (since $\text{InstructorID} \leftrightarrow \text{CourseID}$ forms a bijection). The relation is in 3NF but violates BCNF because FD1 ($\text{CourseID} \rightarrow \text{InstructorID}$) and FD3 ($\text{InstructorID} \rightarrow \text{CourseID}$) have determinants that are not superkeys. CourseID alone and InstructorID alone are not superkeys of the original relation.

Decomposition into BCNF

We decompose using the BCNF-violating FD: $\text{CourseID} \rightarrow \text{InstructorID}$.

Relation 1 - Course_Instructor: (CourseID, InstructorID)

Primary Key: CourseID (also: InstructorID is a candidate key by FD3)

CourseID	InstructorID
DBMS101	I11
OOP201	I12
NET150	I13

Relation 2 - Student_Enrolment: (StudentID, CourseID)

Primary Key: (StudentID, CourseID)

StudentID	CourseID
101	DBMS101
101	OOP201
102	DBMS101
103	NET150
104	OOP201

Why BCNF is satisfied: In Course_Instructor, the only non-trivial FD is $\text{CourseID} \rightarrow \text{InstructorID}$; CourseID is the primary key, so it is a superkey. In Student_Enrolment, the only FD is the trivial one on the composite key (StudentID, CourseID), which is the superkey. Every determinant in both relations is a superkey, satisfying BCNF.

Task 2 - Fourth Normal Form (4NF)

Original Table - Student-Language-Hobby:

StudentID	Language	Hobby
101	English	Chess
101	English	Guitar
101	Spanish	Chess
101	Spanish	Guitar

StudentID	Language	Hobby
102	French	Painting
102	French	Hiking
102	German	Painting
102	German	Hiking

Multivalued Dependencies (MVDs)

MVD1: StudentID $\twoheadrightarrow$ Language

MVD2: StudentID $\twoheadrightarrow$ Hobby

These MVDs hold because a student's set of languages and a student's set of hobbies are completely independent of each other. For every (Language, Hobby) combination of a student, every other language and hobby combination must also appear, which is exactly what we observe in the data.

Why MVDs Violate 4NF

A relation is in 4NF only if for every non-trivial MVD $X \twoheadrightarrow Y$, X is a superkey. Here StudentID is not a superkey of the three-attribute relation (StudentID, Language, Hobby) - the superkey is the full composite (StudentID, Language, Hobby). Therefore, the MVDs StudentID $\twoheadrightarrow$ Language and StudentID $\twoheadrightarrow$ Hobby violate 4NF.

Decomposition into 4NF

We project onto two binary relations, each containing only one MVD:

Relation 1 — Student_Language: (StudentID, Language)

Primary Key: (StudentID, Language)

StudentID	Language
101	English
101	Spanish
102	French
102	German

Relation 2 — Student_Hobby: (StudentID, Hobby)

Primary Key: (StudentID, Hobby)

StudentID	Hobby
101	Chess
101	Guitar
102	Painting
102	Hiking

Each relation now has only a trivial MVD ($\text{StudentID} \twoheadrightarrow \text{Language}$ in Student_Language and $\text{StudentID} \twoheadrightarrow \text{Hobby}$ in Student_Hobby), where the determinant is a superkey of the respective relation. This satisfies 4NF. Adding a new language for a student now requires only one row in Student_Language with no effect on Student_Hobby , eliminating the redundancy.

Task 3 - Fifth Normal Form (5NF)

Original Contract Table:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S2	P1	J1

Join Dependency Identification

The Contract relation contains the following join dependency:

$\text{JD}(\{\text{Supplier, Part}\}, \{\text{Supplier, Project}\}, \{\text{Part, Project}\})$

This means the relation $\text{Contract}(\text{Supplier, Part, Project})$ is losslessly decomposable into three binary projections:

- $\text{SP}(\text{Supplier, Part})$
- $\text{SJ}(\text{Supplier, Project})$
- $\text{PJ}(\text{Part, Project})$

A tuple (S, P, J) exists in Contract if and only if: Supplier S supplies Part P , AND Supplier S works on Project J , AND Part P is used in Project J . We can verify this with the data: $S1$ supplies $P1$ and $P2$; $S1$ works on $J1$ and $J2$; $P1$ is used in $J1$ and $J2$; $P2$ is used in $J1$. Joining $\text{SP} \bowtie \text{SJ} \bowtie \text{PJ}$ regenerates the original four rows exactly.

Why the Table is NOT in 5NF

A relation is in 5NF if every join dependency is implied by the candidate keys. The only candidate key of Contract is the full composite $(\text{Supplier, Part, Project})$. The join dependency $\text{JD}(\{\text{Supplier, Part}\}, \{\text{Supplier, Project}\}, \{\text{Part, Project}\})$ is NOT implied by this candidate key, it is a semantic constraint about the business rules (a supplier participates in supplying a part to a project only when all three binary combinations hold independently). Therefore, the table violates 5NF.

Decomposition into 5NF

We project Contract into three binary relations:

Relation 1 - Supplier_Part (SP): (Supplier, Part)

Primary Key: (Supplier, Part)

Supplier	Part
S1	P1
S1	P2
S2	P1

Relation 2 - Supplier_Project (SJ): (Supplier, Project)

Primary Key: (Supplier, Project)

Supplier	Project
S1	J1
S1	J2
S2	J1

Relation 3 - Part_Project (PJ): (Part, Project)

Primary Key: (Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1

Why 5NF is satisfied: Each binary relation has only its composite primary key as the candidate key, and its only join dependency is trivial (implied by the superkey). The original Contract relation can be reconstructed without loss by the natural join $SP \bowtie SJ \bowtie PJ$, and no further lossless decomposition is possible. This eliminates the redundancy caused by the ternary join dependency.